

Sharif University of Technology
Department of Electrical Engineering

Introduction to Machine Learning

Fall 2025 Semester

Project Phase 2

Instructor: Dr. Sajjad Amini

Group Member 1: Mohammadkazem Shahrabi 401101987

Group Member 2: Farzan Aghaei 401101202

Date: February 18, 2026

Contents

Theoretical Part **2**

1 Question 1: Parameter Efficiency **2**
1.1 Part 1 . . . 2
1.2 Part 2 . . . 2
1.3 Part 3 . . . 2

2 Question 2: The Computational Trick **3**
2.1 Part 1 . . . 3
2.2 Part 2 . . . 3

3 Question 3: Maximum Likelihood and Loss Function **4**
3.1 Part 1 . . . 4
3.2 Part 2 . . . 4

4 Question 4: Sampling vs. Scoring **5**
4.1 Part 1 . . . 5
4.2 Part 2 . . . 5

Practical Part **7**

5 Task 1: Implement NADE **7**

6 Task 2: Training **8**

7 Task 3: OOD Detection **9**

8 Task 4: Generation **10**

Theoretical Part

1 Question 1: Parameter Efficiency

1.1 Part 1

A binary image with D pixels means:

$$x \in \{0,1\}^D$$

So the number of possible distinct images (states) is:

$$|\{0,1\}^D| = 2^D.$$

If we use a fully expressive tabular model, we assign one probability value to each of these 2^D states. But these probabilities must satisfy:

$$\sum_{x \in \{0,1\}^D} p(x) = 1,$$

so one of them is determined by the rest. Therefore the number of free parameters is:

$$\boxed{2^D - 1.}$$

1.2 Part 2

In NADE (as given in the project), we have parameters:

$$W \in \mathbb{R}^{H \times D}, \qquad V \in \mathbb{R}^{D \times H}, \qquad c \in \mathbb{R}^H, \qquad b \in \mathbb{R}^D.$$

So the trainable parameters are:

$$\#(W) = HD, \quad \#(V) = DH, \quad \#(c) = H, \quad \#(b) = D.$$

Adding them gives:

$$HD + DH + H + D = 2HD + H + D.$$

Hence the total number of trainable parameters in NADE is:

$$\boxed{2HD + H + D.}$$

1.3 Part 3

For the tabular approach, the number of parameters grows exponentially with D:

$$\boxed{2^D - 1 \;\; = \;\; \mathcal{O}(2^D).}$$

For NADE, the number of parameters grows linearly in D (assuming hidden size H is fixed):

$$\boxed{2HD + H + D \;\; = \;\; \mathcal{O}(D).}$$

So, the main advantage of NADE is replacing the exponential parameter growth of a fully expressive table with a much more manageable linear scaling in the image dimension.

2 Question 2: The Computational Trick

2.1 Part 1

We are given:

$$a_d = W_{\cdot,<d}\, x_{<d} + c,$$

where $W_{\cdot,<d}$ means the first $d-1$ columns of W, and

$$x_{<d} = [x_1, \ldots, x_{d-1}]^\top.$$

Now look at a_{d+1}:

$$a_{d+1} = W_{\cdot,<d+1}\, x_{<d+1} + c.$$

But $W_{\cdot,<d+1}$ is just $W_{\cdot,<d}$ plus the new column $W_{\cdot,d}$, and

$$x_{<d+1} = [x_{<d}; x_d].$$

So the new term that appears compared to a_d is exactly $W_{\cdot,d}x_d$. Therefore:

$$\boxed{a_{d+1} = a_d + W_{\cdot,d}\, x_d.}$$

2.2 Part 2

To compute $p(x)$ we need all conditionals:

$$p(x) = \prod_{d=1}^{D} p(x_d \mid x_{<d}).$$

For each d, NADE computes:

$$h_d = \sigma(a_d), \qquad \hat{x}_d = \sigma(v_d^\top h_d + b_d),$$

where $v_d^\top$ is the d-th row of V.

With the recursive update Using

$$a_{d+1} = a_d + W_{\cdot,d}x_d,$$

each step updates a with adding a length-H vector, which costs $\mathcal{O}(H)$. Then computing $v_d^\top h_d$ costs $\mathcal{O}(H)$ as well. So each dimension is $\mathcal{O}(H)$, and over D pixels:

$$\boxed{\text{Forward pass complexity} = \mathcal{O}(DH).}$$

Without the recursive update (naive computation) If we did not reuse a_d and for each d computed

$$a_d = W_{\cdot,<d}x_{<d} + c$$

from scratch, then that matrix-vector product involves a matrix of size $H \times (d-1)$, so it costs $\mathcal{O}(H(d-1))$ for step d. Summing over all d:

$$\sum_{d=1}^{D} \mathcal{O}(H(d-1)) = \mathcal{O}\left(H \sum_{d=1}^{D}(d-1)\right) = \mathcal{O}\left(H \cdot \frac{D(D-1)}{2}\right) = \mathcal{O}(HD^2).$$

So:

$$\boxed{\text{Naive forward pass complexity} = \mathcal{O}(HD^2).}$$

In short, the weight-sharing + recursion reduces the forward computation from quadratic (in D) to linear (in D).

3 Question 3: Maximum Likelihood and Loss Function

3.1 Part 1

We start from the chain rule factorization of NADE:

$$p(x) = \prod_{d=1}^{D} p(x_d \mid x_{<d}).$$

Taking the logarithm:

$$\log p(x) = \sum_{d=1}^{D} \log p(x_d \mid x_{<d}).$$

Since $x_d \in \{0, 1\}$ and NADE models

$$\hat{x}_d = p(x_d = 1 \mid x_{<d}),$$

each conditional is a Bernoulli distribution:

$$p(x_d \mid x_{<d}) = \hat{x}_d^{x_d}(1 - \hat{x}_d)^{1-x_d}.$$

Therefore:

$$\log p(x_d \mid x_{<d}) = x_d \log \hat{x}_d + (1 - x_d) \log(1 - \hat{x}_d).$$

Substituting back:

$$\log p(x) = \sum_{d=1}^{D} \left[x_d \log \hat{x}_d + (1 - x_d) \log(1 - \hat{x}_d) \right].$$

The negative log-likelihood (NLL) is:

$$-\log p(x) = -\sum_{d=1}^{D} \left[x_d \log \hat{x}_d + (1 - x_d) \log(1 - \hat{x}_d) \right].$$

Hence minimizing the NLL is exactly equivalent to minimizing the sum of Binary Cross Entropy (BCE) losses over pixels:

$$\boxed{\mathcal{L}(x) = -\sum_{d=1}^{D} \left[x_d \log \hat{x}_d + (1 - x_d) \log(1 - \hat{x}_d) \right].}$$

3.2 Part 2

In standard NADE, each conditional $p(x_d \mid x_{<d})$ is modeled as a Bernoulli distribution. This assumes:

$$x_d \in \{0, 1\}.$$

That is why we binarize MNIST: after thresholding at 0.5, each pixel can be interpreted as a Bernoulli random variable (black or white). This makes the probabilistic model consistent with the BCE loss derived above.

If we instead used continuous values in $[0, 1]$ with the same formulation, we would still plug them into:

$$x_d \log \hat{x}_d + (1 - x_d) \log(1 - \hat{x}_d),$$

but this would no longer correspond to the log-likelihood of a valid Bernoulli random variable. In that case, the model would be trained with BCE as a surrogate loss, but the probabilistic interpretation as a proper generative model for continuous pixel intensities would break down.

So binarization ensures that the likelihood function and the loss function are fully consistent with the assumed Bernoulli data model.

4 Question 4: Sampling vs. Scoring

4.1 Part 1

To generate a new image x_{new} from a trained NADE model, we use the chain rule factorization:

$$p(x) = \prod_{d=1}^{D} p(x_d \mid x_{<d}).$$

Sampling must follow this order because each pixel depends on the previously generated ones.

We start with an empty image and initialize:

$$a_1 = c.$$

Then for $d = 1, 2, \ldots, D$ we repeat:

First compute the hidden activation:

$$h_d = \sigma(a_d).$$

Then compute the conditional probability:

$$\hat{x}_d = \sigma(v_d^\top h_d + b_d).$$

Now we sample the pixel value from a Bernoulli distribution:

$$x_d \sim \text{Bernoulli}(\hat{x}_d).$$

After sampling x_d, we update the pre-activation recursively:

$$a_{d+1} = a_d + W_{\cdot,d} x_d.$$

After completing this process for all D pixels, we obtain:

$$\boxed{x_{\text{new}} = (x_1, \ldots, x_D).}$$

This procedure generates one full image in raster order.

4.2 Part 2

To compute the probability of an existing image x, we do not need to sample. All pixel values are already known, so we only evaluate:

$$p(x) = \prod_{d=1}^{D} p(x_d \mid x_{<d}),$$

or equivalently the log-likelihood.

Although the model is autoregressive, when scoring we simply propagate forward using the given x_d values. There is no randomness involved and no branching computation. For a batch of images, all matrix operations (multiplications, sigmoid evaluations, etc.) can be vectorized across the batch dimension. This allows efficient parallel computation on GPU.

In contrast, sampling is strictly sequential. At step d, we cannot compute x_{d+1} before sampling x_d, because the recursive update

$$a_{d+1} = a_d + W_{\cdot,d} x_d$$

requires the sampled value. Therefore sampling must proceed pixel by pixel.

So even though both scoring and sampling have linear complexity in D, scoring can be parallelized across many images and uses deterministic operations, while sampling is inherently sequential and therefore slower in practice. “

Practical Part

5 Task 1: Implement NADE

In this part, we implemented the Neural Autoregressive Distribution Estimator (NADE) for modeling binary MNIST images. Since each image is represented as a binary vector $x \in \{0,1\}^{784}$, the model factorizes the joint distribution using the autoregressive decomposition:

$$p(x) = \prod_{d=1}^{D} p(x_d \mid x_{<d}).$$

Model Parameters The model contains four trainable parameters:

$$W \in \mathbb{R}^{H \times D}, \quad V \in \mathbb{R}^{D \times H}, \quad c \in \mathbb{R}^{H}, \quad b \in \mathbb{R}^{D},$$

where H is the hidden dimension and $D = 784$ is the number of pixels. The matrices W and V correspond to input-to-hidden and hidden-to-output connections, respectively, while c and b are bias terms.

Weights were initialized with small random values and biases were initialized to zero.

Forward Pass (Log-Likelihood Computation) Instead of computing each conditional independently, we used the recursive update derived in Question 2:

$$a_{d+1} = a_d + W_{\cdot,d} x_d.$$

This avoids recomputing matrix multiplications from scratch at every step and reduces the complexity from $\mathcal{O}(HD^2)$ to $\mathcal{O}(HD)$.

At each step d, we compute:

$$h_d = \sigma(a_d),$$
$$\hat{x}_d = \sigma(v_d^\top h_d + b_d),$$

and accumulate the log-probability using the Bernoulli log-likelihood formulation. For numerical stability, we used `binary_cross_entropy_with_logits` to compute the negative log-likelihood directly from logits.

A simplified version of the core recursive computation is shown below:

```
a = c.expand(batch_size, H)
for d in range(D):
    h = torch.sigmoid(a)
    logits_d = (h * V[d]).sum(dim=1) + b[d]
    logp -= F.binary_cross_entropy_with_logits(
        logits_d, x[:, d], reduction="none"
    )
    a = a + x[:, d].unsqueeze(1) * W[:, d]
```

This function returns $\log p(x)$ for each image in the batch.

Sampling Procedure To generate new images, we implemented a sequential sampling method following the autoregressive factorization. Starting from $a_1 = c$, we iteratively sample:

$$x_d \sim \text{Bernoulli}(\hat{x}_d),$$

and update the hidden pre-activation using the same recursive rule.

Since each pixel depends on previously sampled pixels, generation must be performed strictly in raster order and cannot be parallelized across dimensions.

Overall, the implementation directly follows the theoretical formulation of NADE and efficiently exploits weight sharing and recursive computation to scale linearly with the image dimension.

6 Task 2: Training

In this part, we trained the NADE model on the in-distribution (ID) training set consisting of digits 0–4. The hidden dimension was set to $H = 500$ and the model was optimized using the Adam optimizer with learning rate 10^{-3}. The objective function was the negative log-likelihood (NLL), averaged over the batch.

Training Behavior Figure 1 shows the average training and validation NLL over 20 epochs.

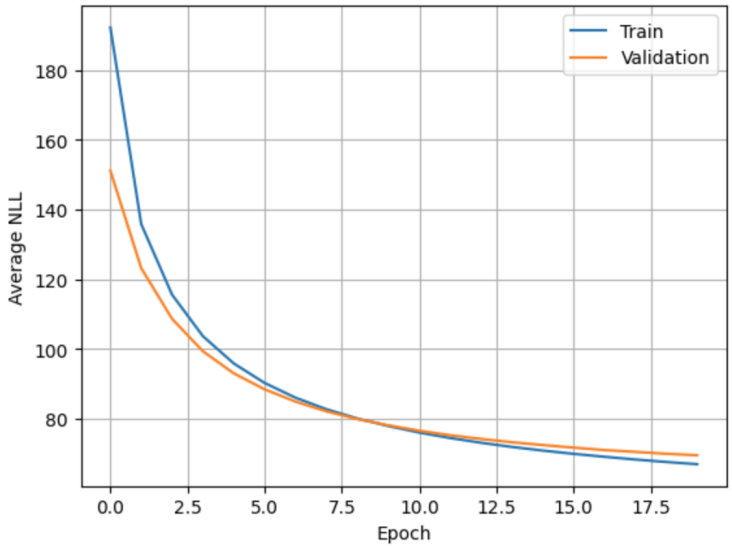


Figure 1: Average training and validation NLL over epochs.

At the beginning of training, the NLL values are relatively large (around 192 for training and 151 for validation), which is expected since the model parameters are randomly initialized. During the first few epochs, the loss decreases rapidly, indicating that the model quickly learns basic structural patterns of the digits. After approximately 8–10 epochs, the decrease becomes more gradual and the curve starts to stabilize.

By the final epoch, the training NLL reached approximately:

$$\text{Train NLL} \approx 66.91,$$

and the validation NLL reached:

$$\text{Validation NLL} \approx 69.48.$$

Generalization and Overfitting An important observation is that the training and validation curves remain close to each other throughout training. The validation loss consistently follows the same decreasing trend as the training loss, and there is no noticeable divergence between the two curves. This indicates that the model does not suffer from significant overfitting, despite having a relatively large hidden dimension ($H = 500$).

The small gap between training and validation NLL (around 2–3 units at the end) suggests good generalization on unseen ID samples. The smooth and monotonic decrease of both curves also indicates stable optimization behavior under Adam.

Conclusion Overall, the training process was stable and convergent. The NADE model successfully minimized the negative log-likelihood on the ID dataset, and the validation results confirm that the learned autoregressive distribution generalizes well to unseen samples from the same distribution.

7 Task 3: OOD Detection

In this part, we evaluated the trained NADE model for out-of-distribution (OOD) detection. Following the probabilistic formulation, we used the log-likelihood $\log p(x)$ as an anomaly score. Since the model was trained only on digits 0–4, we expect higher log-likelihood values for in-distribution (ID) samples and lower values for OOD samples (digits 5–9).

Average Log-Likelihood The average log-likelihood values on the test sets are:

$$\text{Average log-likelihood (ID test)} \approx -68.13,$$
$$\text{Average log-likelihood (OOD test)} \approx -99.51.$$

The ID samples receive significantly higher likelihood (less negative values) compared to OOD samples. This confirms that the model assigns lower probability to unseen digit classes, which is the expected behavior of a generative density model.

Histogram Analysis Figure 2 shows the overlaid histograms of the log-likelihoods for ID and OOD samples.

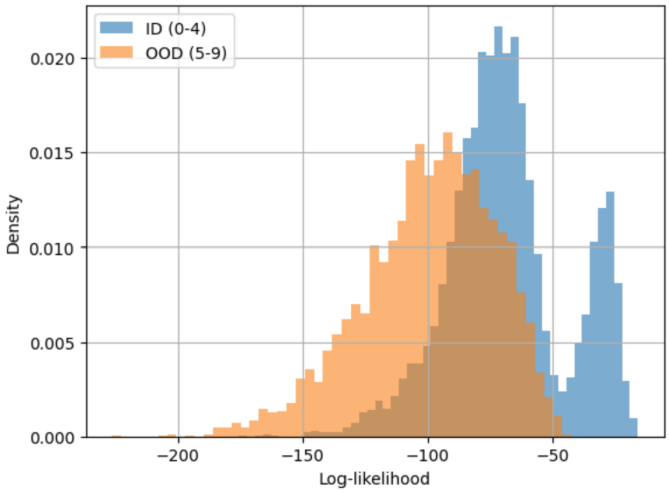


Figure 2: Histogram of log-likelihood values for ID (0–4) and OOD (5–9) samples.

From the histogram, we observe that the ID distribution is shifted to the right (higher log-likelihood), while the OOD distribution is shifted to the left (lower log-likelihood). Although there is some overlap between the two distributions, the separation is clearly visible. This overlap is expected, since some OOD digits may share local pixel structures with ID digits.

AUROC Metric To quantitatively evaluate detection performance, we computed the AUROC (Area Under the ROC Curve), labeling OOD samples as 1 and using $-\log p(x)$ as the anomaly score. The resulting value is:

$$\text{AUROC} \approx 0.807.$$

An AUROC of 0.5 corresponds to random guessing, while 1.0 indicates perfect separation. Therefore, a value around 0.81 demonstrates that the NADE model provides meaningful discriminative power for distinguishing ID from OOD samples, even though it was not explicitly trained for classification.

Conclusion Overall, the likelihood-based OOD detection works reasonably well. The clear shift in average log-likelihood and the AUROC above 0.8 indicate that the autoregressive density model successfully captures the distribution of digits 0–4 and assigns lower probability to unseen digit classes.

8 Task 4: Generation

In this part, we used the trained NADE model to generate new binary images by sampling pixel-by-pixel according to the autoregressive factorization. Because NADE models:

$$p(x) = \prod_{d=1}^{D} p(x_d \mid x_{<d}),$$

the sampling process must be performed sequentially: at each step we first compute $\hat{x}_d = p(x_d = 1 \mid x_{<d})$, then sample $x_d \sim \text{Bernoulli}(\hat{x}_d)$, and continue to the next pixel.

Sampling Method We generated 10 samples using the implemented `sample` function. The key point is that generation cannot be parallelized across pixels because x_d is required before computing $p(x_{d+1} \mid x_{\leq d})$. This is also the reason sampling is slower than likelihood evaluation.

Generated Samples and Discussion Figure 3 shows the generated images.

Since the model was only trained on digits 0–4, we expect the generated samples to mostly resemble these digits rather than digits 5–9. This is consistent with the observed results: several samples clearly look like valid digits from the training classes (for example, shapes similar to 1, 2, 3, 4, and 0). This shows that the model has learned meaningful local dependencies between pixels and is not producing purely random noise.

At the same time, some samples contain small artifacts such as disconnected strokes or isolated white pixels. This behavior is expected for standard NADE on binarized images, because: (i) the model is not convolutional and therefore does not explicitly encode spatial locality, and (ii) the raster-order autoregressive dependency makes long-range structure harder to represent. Overall, the samples are recognizable and class-consistent, but not perfectly clean.

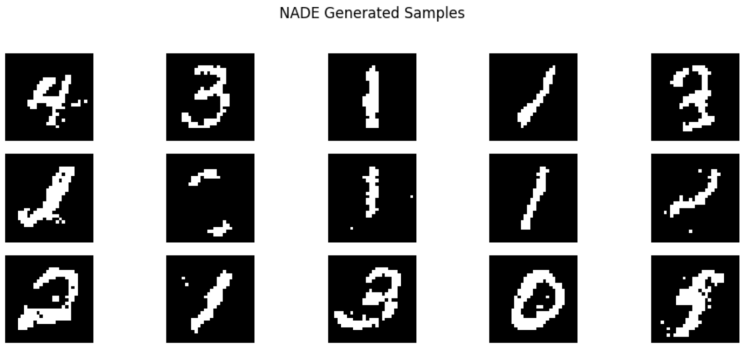


Figure 3: Ten samples generated by NADE after training on digits 0–4.

Conclusion The generation results indicate that NADE successfully captures the distribution of digits 0–4 at a coarse level, producing digit-like shapes instead of random noise. However, the presence of minor artifacts suggests that the model's capacity and architecture limit the sharpness and global consistency of the generated images.